SECURITY AUDIT REPORT

NetworkLearn CCNA Platform

Full-Stack Web Application Security Assessment

Target: https://networklearn.wasmer.app

Supabase Project: jhesstimsojwmkdysmpy

Audit Date: August 5, 2026

Methodology: OWASP Testing Guide v4 + Manual Penetration Testing

Report Version: 2.0 (Post Re-Verification)

CONFIDENTIAL

Table of Contents

1. Executive Summary

This report presents the findings of a comprehensive security audit conducted against the NetworkLearn CCNA Platform (`https://networklearn.wasmer.app`), a web application built with Astro v7, React 19, and Supabase. The audit covered external reconnaissance, static code review, and penetration testing across three user personas: anonymous, regular authenticated user, and administrator.

Following an initial audit, a re-verification cycle was performed on August 5, 2026. One finding (Service Role Key in localStorage) has been fully remediated, and two findings (Hardcoded Credentials, Incomplete SignOut) have been partially addressed. All other findings remain valid.

> **Overall Risk Rating: CRITICAL** -- Multiple privilege escalation vectors, stored XSS vulnerabilities, and missing server-side access controls allow any authenticated user to gain administrative privileges and compromise the platform.

Total Valid Findings: 26 (excluding 1 fully-fixed and 2 partially-fixed findings)

Key Findings at a Glance

Privilege Escalation: Any user can self-promote to admin and self-upgrade to Pro plan via direct Supabase API calls (RLS UPDATE policy on `profiles` table).

Missing Server-Side Auth: Admin page HTML is served to all visitors; authorization is client-side only.

Stored XSS: Unsanitized `display_name` and logo URLs are injected via `innerHTML` in the admin panel.

Quiz Integrity: Correct answers are shipped in the client bundle, and scores are calculated client-side without server validation.

Sensitive Data Exposure: API keys, user emails, and avatars persist in `localStorage` and are not cleared on sign-out.

Missing Security Headers: No CSP, HSTS, X-Frame-Options, or other protective HTTP headers are configured.

2. Scope & Methodology

Target Application

Property	Value
Application URL	`https://networklearn.wasmer.app`
Supabase Project	`jhesstimsojwmkdysmpy`
Framework	Astro v7 + React 19
Backend	Supabase (PostgreSQL, Auth, RLS, RPC)
Hosting	Wasmer Edge

OWASP Categories Tested

OWASP Top 10 2021	Coverage
A01 -- Broken Access Control	Full (RLS, auth checks, IDOR, privilege escalation)
A02 -- Cryptographic Failures	Full (localStorage exposure, key management)
A03 -- Injection	XSS via innerHTML, input validation
A04 -- Insecure Design	Feature flags, quiz scoring, account deletion flow
A05 -- Security Misconfiguration	HTTP headers, endpoints, admin email exposure
A06 -- Vulnerable Components	npm audit (pass)
A07 -- Authentication Failures	Email verification, signOut, credential management

Test Personas

Persona	Context	Scope
Anonymous	Unauthenticated visitor	External recon, unauthenticated access, endpoint probing
Regular User	Authenticated non-admin user	Privilege escalation, XSS, IDOR, data tampering
Admin User	Authenticated administrator	Admin panel security, RPC validation, RLS completeness

Methodology

The audit followed the **OWASP Testing Guide v4** and included:

External reconnaissance (headers, endpoints, JavaScript bundles, HTTP responses)
Static code analysis of the full source tree (src/, supabase/migrations/, config files)

Dynamic penetration testing across all three personas
RLS policy inventory and completeness verification
Post-remediation re-verification cycle

3. Findings Summary Matrix

#	Finding	Severity	OWASP	Status
1	Self-Promote to Admin via Profile UPDATE	CRITICAL	A01	Valid
2	Self-Upgrade to Pro Plan Without Payment	CRITICAL	A01	Valid
3	Admin Page Served Without Server-Side Auth	CRITICAL	A01	Valid
4	Admin Email Exposed in Client-Side JavaScript	CRITICAL	A04	Valid
5	Stored XSS via display_name in Admin Panel (innerHTML)	HIGH	A03	Valid
6	Stored XSS via Logo URL in innerHTML	HIGH	A03	Valid
7	Feature Flags Defined But Never Enforced	HIGH	A04	Valid
8	Avatar Stored as Base64 in Database Column	HIGH	A04	Valid
9	Sensitive User Data Stored in localStorage	HIGH	A05	Valid
10	Real-time Subscription IDOR via localStorage userId	HIGH	A01	Valid
11	Incomplete Sign-Out -- Sensitive Data Remains	HIGH	A07	Partial Fix
12	Missing All Critical HTTP Security Headers	HIGH	A05	Valid
13	No Email Verification Enforcement After Signup	MEDIUM	A07	Valid
14	Client-Side Admin Authorization with DOM-Based Access Control	MEDIUM	A01	Valid
15	Hardcoded Admin Email in Multiple Locations	MEDIUM	A05	Valid
16	Display Name Input Has No Validation	MEDIUM	A03	Valid
17	Account Deletion Not Using the Database RPC	MEDIUM	A04	Valid
18	Client-Side Quiz Score Calculation Allows Tampering	MEDIUM	A04	Valid
19	Client-Side Lesson Progress Can Be Faked via localStorage	MEDIUM	A04	Valid
20	increment_points RPC Lacks Bounds Check	MEDIUM	A04	Valid
21	Revolut API Key Stored in Client-Side localStorage	MEDIUM	A02	Valid
22	delete_user_account GRANT Too Broad	LOW	A01	Valid
23	Client-Side-Only Password Validation	LOW	A07	Valid

#	Finding	Severity	OWASP	Status
24	Admin Settings Reinitialized with Hardcoded Defaults	LOW	A04	Valid
25	email_encrypted Column Stores Plaintext	LOW	A04	Valid
26	No security.txt File	LOW	A05	Valid
27	No robots.txt or sitemap.xml	LOW	A05	Valid

Note: One finding (Service Role Key in localStorage) was fully remediated and has been excluded from this report. Two findings (Hardcoded Credentials, Incomplete SignOut) are marked as Partial Fix.

4. Detailed Findings by OWASP Category

4.1 -- A01: Broken Access Control

CRITICAL **F-01: Self-Promote to Admin via Profile UPDATE** A01 -- Broken Access Control

DESCRIPTION

The Supabase RLS policy on the `profiles` table allows any authenticated user to UPDATE their own row, including the `is_admin` column. There is no server-side restriction preventing a user from setting `is_admin = true` on their own profile.

EVIDENCE

Client-side code (Header.tsx):

```
await supabase.from('profiles').update({
  is_admin: true
}).eq('id', session.user.id);
```

RLS policy (migration 001):

```
-- Users can update own profile
CREATE POLICY "Users can update own profile"
ON profiles FOR UPDATE
USING (auth.uid() = id);
```

IMPACT

Any authenticated user can gain full administrator privileges, including access to user management, subscription controls, and API key management. This is a complete compromise of the authorization model.

REMEDIATION

Replace the broad UPDATE policy with a trigger or RPC that prevents modification of the `is_admin` column by non-admin users. Only allow admin updates through the `is_admin()` security definer function.

Effort: Small (1-2 hours)

CRITICAL

F-02: Self-Upgrade to Pro Plan Without Payment

A01 -- Broken Access Control

DESCRIPTION

The same RLS UPDATE policy on `profiles` allows users to modify the `plan` column. A user can set `plan = 'pro'` on their own profile to gain premium features without completing payment.

EVIDENCE

```
await supabase.from('profiles').update({
  plan: 'pro'
}).eq('id', session.user.id);
```

IMPACT

Bypass of all paid subscription features. Potential revenue loss. The same RLS gap enables both privilege escalation and plan bypass.

REMEDIATION

Same as F-01. The `plan` column should be immutable by the client. Only the `is_admin()` function or a payment webhook should update it.

Effort: Small (1-2 hours)

CRITICAL **F-03: Admin Page Served Without Server-Side Authentication** A01 -- Broken Access Control

DESCRIPTION

The admin page at `/en/admin` returns HTTP 200 with the full admin dashboard HTML to any unauthenticated request. The page contains user management tables, subscription controls, certificate generation buttons, and settings panels for Revolut API keys, Supabase service keys, and Stripe. No server-side middleware or authentication check exists.

EVIDENCE

Request:

```
GET /en/admin HTTP/1.1
Host: networklearn.wasmer.app
```

Response: HTTP 200 with full admin dashboard HTML. No redirect to login. The client-side JavaScript uses CSS classes (`hidden`) to show/hide content, but the HTML structure is fully accessible.

IMPACT

Complete information disclosure of admin interface structure. Attackers can study the full admin UI, identify all form fields, and craft targeted DOM manipulation attacks to bypass client-side checks.

REMEDIATION

Implement Astro middleware or server-side authentication checks that return HTTP 403 or redirect to login for unauthenticated users. Do not serve admin HTML to non-admin visitors.

Effort: Medium (3-5 hours)

HIGH

F-10: Real-time Subscription IDOR via localStorage userId

A01 -- Broken Access Control (IDOR)

DESCRIPTION

The dashboard page reads the user ID from `localStorage` to create real-time Supabase subscriptions. A user can modify their `localStorage` to listen to another user's activity.

EVIDENCE

```
// dashboard.astro (lines 533–574)
const userId = localStorage.getItem('userId');
if (userId) {
  supabase
    .channel('lesson-progress-changes')
    .on('postgres_changes', {
      event: '*',
      schema: 'public',
      table: 'lesson_progress',
      filter: `user_id=eq.${userId}`,
    }, () => { loadDashboard(); })
    .subscribe();
}
```

Attack scenario:

```
localStorage.setItem('userId', '<target-user-uuid>');
location.reload();
```

IMPACT

Information disclosure of another user's learning activity in real-time. *Note:* Mitigated by RLS policies (Supabase applies policies server-side), so this is not directly exploitable but remains a defense-in-depth concern.

REMEDIATION

Use the Supabase auth session (`supabase.auth.getSession()`) to derive the userId server-side rather than reading from localStorage.

Effort: Small (1-2 hours)

MEDIUM F-14: Client-Side Admin Authorization with DOM-Based Access Control

A01 -- Broken Access Control

DESCRIPTION

The admin page renders all admin content in the HTML and uses CSS classes (`hidden`) to show/hide it based on JavaScript authorization checks. The full admin HTML is delivered to every visitor.

EVIDENCE

```
// src/pages/en/admin.astro (lines 395–418)
// Content is hidden via CSS, not server-side rendering
<div class="hidden" id="admin-panel">
  <!-- Full admin UI rendered here -->
</div>
```

IMPACT

Information disclosure of admin email and interface structure. Any visitor can inspect the DOM to see admin content.

REMEDIATION

Move admin authorization to middleware or server-side rendering that returns a 403 response for non-admin users.

Effort: Medium (3-5 hours)

LOW F-22: delete_user_account GRANT Too Broad

A01 -- Broken Access Control

DESCRIPTION

The `delete_user_account` RPC function is granted to the `authenticated` role, meaning any logged-in user can call it. Internal checks prevent deleting other users, but the broad GRANT increases attack surface.

EVIDENCE

```
-- supabase/migrations/006_delete_account.sql (line 29)
GRANT EXECUTE ON FUNCTION delete_user_account TO authenticated;
```

REMEDIATION

Restrict the GRANT to specific roles or use the `delete_own_account` RPC function from migration 007 which has tighter internal checks.

Effort: Trivial (30 min)

4.2 -- A02: Cryptographic Failures

HIGH **F-09: Sensitive User Data Stored in localStorage** A05 -- Security Misconfiguration

DESCRIPTION

Multiple sensitive data items are stored in `localStorage`, which is accessible to any JavaScript on the page. Without a Content Security Policy (CSP), any XSS vulnerability can exfiltrate all stored data.

EVIDENCE

```
localStorage.setItem('userEmail', e.user.email || '');
localStorage.setItem('userId', e.user.id);
localStorage.setItem('adminSettings', JSON.stringify({...}));
localStorage.setItem('userAvatar', base64DataUrl);
```

IMPACT

localStorage is accessible to any script on the page. Combined with the XSS findings below, an attacker can exfiltrate user emails, IDs, admin settings, and API keys.

REMEDIATION

Use httpOnly cookies for sensitive session data. Implement CSP. Minimize localStorage usage for sensitive data. Store admin settings server-side.

Effort: Medium (4-6 hours)

MEDIUM

F-21: Revolut API Key Stored in Client-Side localStorage

A02 -- Cryptographic Failures

DESCRIPTION

The pricing page reads the Revolut API key from `localStorage` and uses it directly in a browser fetch call. Payment API keys must never be handled client-side.

EVIDENCE

```
// src/pages/en/pricing.astro (lines 198–200)
const settings = JSON.parse(localStorage.getItem('adminSettings') || '{}');
const revolutApiKey = settings.revolutApiKey;
const response = await fetch('https://merchant.revolut.com/api/1.0/orders', {
  headers: { 'Authorization': `Bearer ${revolutApiKey}` },
});
```

IMPACT

Revolut API key compromise could allow unauthorized payment operations, order manipulation, and financial data access.

REMEDIATION

Never handle payment API keys client-side. All Revolut API calls should go through a server-side edge function or Supabase Edge Function.

Effort: Medium (3-5 hours)

4.3 -- A03: Injection (Cross-Site Scripting)

HIGH **F-05: Stored XSS via display_name in Admin Panel (innerHTML)** A03 -- Injection

DESCRIPTION

The admin panel renders user `display_name` values using `innerHTML` without sanitization. A user can set their display name to a malicious script that executes in the admin's browser when the admin views the user management table.

EVIDENCE

```
// src/pages/en/admin.astro (lines 498, 512, 631)
// display_name interpolated into innerHTML without sanitization
td.textContent = profile.display_name;  // Some locations use textContent
// BUT other locations use innerHTML with user data
```

Re-verification confirms: `display_name` is still interpolated into `innerHTML` in admin.astro (lines 498, 512, 631) without sanitization.

IMPACT

Full account takeover of administrator. The attacker can steal the admin's session, read all user data, modify subscriptions, and access API keys stored in `localStorage`.

REMEDIATION

Use `textContent` instead of `innerHTML` for user-supplied data. If HTML rendering is required, use a DOMPurify sanitizer. Add input validation on `display_name` (max 50 chars, no HTML tags).

Effort: Small (1-2 hours)

HIGH **F-06: Stored XSS via Logo URL in innerHTML** A03 -- Injection

DESCRIPTION

The logo preview function injects a user-controlled URL directly into an `innerHTML` assignment with an `onerror` handler, enabling XSS via crafted URLs.

EVIDENCE

```
// src/pages/en/admin.astro (line 795)
preview.innerHTML = `<img src="${logoUrl}" alt="Logo"
  class="w-10 h-10 rounded-xl object-cover"
  onerror="this.parentElement.innerHTML='N'" />`;
```

IMPACT

An attacker can craft a logo URL that breaks out of the `src` attribute and injects arbitrary JavaScript via the `onerror` handler or attribute escape.

REMEDIATION

```
// Use DOM APIs instead of innerHTML:
const img = document.createElement('img');
img.src = logoUrl;
img.onerror = () => { preview.textContent = 'N'; };
preview.replaceChildren(img);
```

Effort: Trivial (30 min)

MEDIUM

F-16: Display Name Input Has No Validation

A03 -- Injection

DESCRIPTION

The profile form updates `display_name` directly from the input field value with no client-side or server-side validation. No `maxLength`, `minLength`, or pattern constraints exist. No database CHECK constraint is applied.

EVIDENCE

```
// src/pages/en/profile.astro (lines 470–474)
await supabase.from('profiles').update({
  display_name: displayNameInput.value
}).eq('id', session.user.id);
```

IMPACT

Enables the XSS attack in F-05. A user could set an extremely long display name causing performance issues, or inject HTML/JavaScript content.

REMEDIATION

Add client-side validation (max 50 chars) and a database CHECK constraint: `CHECK (length(display_name) <= 50)`.

Effort: Trivial (30 min)

4.4 -- A04: Insecure Design

CRITICAL **F-04: Admin Email Exposed in Client-Side JavaScript** A04 -- Insecure Design

DESCRIPTION

The admin email address is hardcoded as a string literal in the compiled JavaScript bundle. This exposes the exact admin identity to anyone inspecting the client code.

EVIDENCE

```
// _astro/Header.D-7z5LJB.js
var s = 'tsiartasantreas@gmail.com';
// ...
p(e.user.email === s)
```

IMPACT

Enables targeted phishing and social engineering attacks against the administrator.

REMEDIATION

Remove hardcoded email from client-side code. Use the `profiles.is_admin` boolean column exclusively for admin checks. Admin assignment should be done via direct database update only.

Effort: Small (1-2 hours)

HIGH

F-07: Feature Flags Defined But Never Enforced

A04 -- Insecure Design

DESCRIPTION

Feature flags `ENABLE_REGISTRATION`, `ENABLE_PASSWORD_RESET`, `ENABLE_GOOGLE_OAUTH`, and `MAINTENANCE_MODE` are defined in `config.ts` but are never imported or checked anywhere in the codebase.

EVIDENCE

```
// src/lib/config.ts (lines 34-37)
export const FEATURES = {
  ENABLE_REGISTRATION: true,
  ENABLE_PASSWORD_RESET: true,
  ENABLE_GOOGLE_OAUTH: false,
  MAINTENANCE_MODE: false,
};
```

Re-verification confirms: These flags are never imported or checked in any page.

IMPACT

An admin who disables registration via the admin panel believes it is enforced, but it is not. Users can still register. Google OAuth remains clickable even when disabled.

REMEDIATION

Each page that uses these flags must import and check them before rendering protected UI. Alternatively, enforce flags server-side via middleware.

Effort: Small (2-3 hours)

HIGH

F-08: Avatar Stored as Base64 in Database Column

A04 -- Insecure Design

DESCRIPTION

User avatars are converted to base64 data URLs and stored directly in the `profiles.avatar_url` column. Every query to `profiles` returns the full base64 string, creating massive payloads.

EVIDENCE

```
// src/pages/en/profile.astro (lines 325-350)
const reader = new FileReader();
reader.onload = async (e) => {
  const base64 = e.target.result;
  await supabase.from('profiles').update({
    avatar_url: base64  // 2MB image = ~2.7MB base64
  }).eq('id', session.user.id);
};
```

IMPACT

The admin panel loads ALL profiles including all base64 avatars. This creates massive page payloads, excessive database storage costs, and potential DoS via large uploads.

REMEDIATION

Use Supabase Storage for file uploads and store only the public URL in the database. Set file size limits and validate image types server-side.

Effort: Medium (3-5 hours)

MEDIUM

F-15: Hardcoded Admin Email in Multiple Locations

A05 -- Security Misconfiguration

DESCRIPTION

The admin email `tsiartasantreas@gmail.com` appears in 4+ client-side locations including `src/lib/admin.ts`, `src/pages/en/admin.astro`, `src/pages/en/login.astro`, and multiple SQL migrations.

EVIDENCE

```
// src/lib/admin.ts (line 2)
const ADMIN_EMAIL = 'tsiartasantreas@gmail.com';

// src/pages/en/admin.astro (line 38)
const isAdmin = user.email === 'tsiartasantreas@gmail.com';

// src/pages/en/login.astro (line 142)
if (user.email === 'tsiartasantreas@gmail.com') { ... }
```

REMEDIATION

Consolidate admin check to use `profiles.is_admin` column. Remove all hardcoded email references from client code.

Effort: Small (1-2 hours)

MEDIUM F-17: Account Deletion Not Using the Database RPC

A04 -- Insecure Design

DESCRIPTION

A proper `delete_user_account` RPC function exists but is never called. The profile page performs individual table deletes instead, leaving the `auth.users` record intact.

EVIDENCE

```
// src/pages/en/profile.astro (lines 439–453)
// Does individual deletes:
await supabase.from('profiles').delete().eq('id', userId);
await supabase.from('lesson_progress').delete().eq('user_id', userId);
await supabase.from('quiz_scores').delete().eq('user_id', userId);
await supabase.from('user_points').delete().eq('user_id', userId);
// NEVER calls: supabase.rpc('delete_user_account', ...)
```

IMPACT

The user's `auth.users` record is never deleted, meaning the email remains registered and the account can be re-activated.

REMEDIATION

Replace individual deletes with `supabase.rpc('delete_own_account')` from migration 007, which handles all table deletes and the auth.users deletion atomically.

Effort: Trivial (30 min)

MEDIUM

F-18: Client-Side Quiz Score Calculation Allows Tampering

A04 -- Insecure Design

DESCRIPTION

Quiz scores are calculated entirely on the client in `QuizEngine.tsx`. The calculated score is sent directly to the database without server-side verification. A `save_quiz_score` RPC exists in migration 007 but is NOT called.

EVIDENCE

```
// src/components/quiz/QuizEngine.tsx (lines 69-85)
const calculateScore = useCallback(() => {
  let correct = 0;
  questions.forEach((q, idx) => {
    if (selectedAnswers[idx] === q.correct) {
      correct++;
    }
  });
  const percentage = Math.round((correct / questions.length) * 100);
  return { correct, total: questions.length, percentage, ... };
}, [questions, selectedAnswers]);
```

Re-verification confirms: Client does direct INSERT into `quiz_scores` with fabricated scores. The `save_quiz_score` RPC exists but is not called.

IMPACT

Users can artificially inflate quiz scores to earn certificates without completing the material. The learning platform's certification integrity is compromised.

REMEDIATION

Move score calculation to the `save_quiz_score` RPC function. The client should submit selected answers, not the score. The server validates answers against the database.

Effort: Medium (3-5 hours)

MEDIUM **F-19: Client-Side Lesson Progress Can Be Faked via localStorage** A04 -- Insecure Design

DESCRIPTION

Lesson completion status is stored in `localStorage` with keys like `completedLessons_{userId}`. A user can modify these values to mark lessons as complete without actually completing them.

EVIDENCE

```
// Lesson progress stored in localStorage
localStorage.setItem(`completedLessons_${userId}`,
  JSON.stringify([1, 2, 3, 4, 5]));
```

IMPACT

Users can bypass learning requirements and claim course completion without engaging with the material.

REMEDIATION

Use the `complete_lesson` RPC function from migration 007 for server-validated lesson completion. Read progress from the database, not localStorage.

Effort: Small (2-3 hours)

MEDIUM **F-20: increment_points RPC Lacks Bounds Check** A04 -- Insecure Design

DESCRIPTION

The `increment_points` RPC correctly validates that `auth.uid() == p_user_id`, but accepts any arbitrary `p_points` value from the client. Users can call the RPC with very large numbers.

EVIDENCE

```
-- supabase/migrations/004_security_fixes.sql (lines 6-19)
IF auth.uid() != p_user_id THEN
  RAISE EXCEPTION 'Cannot modify another user''s points';
END IF;
-- No validation on p_points value
UPDATE user_points
SET total_points = total_points + p_points
WHERE user_id = p_user_id;
```

IMPACT

A user could call `increment_points` with a very large number to artificially inflate their points, potentially qualifying for rewards or leaderboards.

REMEDIATION

```
IF p_points < 0 OR p_points > 100 THEN
  RAISE EXCEPTION 'Points must be between 0 and 100';
END IF;
```

Effort: Trivial (30 min)

4.5 -- A05: Security Misconfiguration

HIGH **F-12: Missing All Critical HTTP Security Headers** A05 -- Security Misconfiguration

DESCRIPTION

No security-related HTTP headers are configured. The application is vulnerable to clickjacking, MIME sniffing, HTTP downgrade attacks, and script injection.

EVIDENCE

```
HTTP/2 200
content-length: 122
content-type: text/html
cache-control: public, max-age=86400

-- Missing:
Content-Security-Policy
X-Frame-Options
X-Content-Type-Options
Strict-Transport-Security
Referrer-Policy
Permissions-Policy

-- Exposed:
x-wasmer-request-id: (internal tracking)
x-edge-app-version-id: dav_nX3IVt3uQ645 (deployment version)
```

IMPACT

Vulnerable to clickjacking (site can be framed), MIME sniffing, HTTP downgrade attacks (no HSTS), and arbitrary script injection (no CSP).

REMEDIATION

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' https://font
s.googleapis.com; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src
'self' https://fonts.gstatic.com; img-src 'self' data: https:; connect-src 'self' https://j
hesstimsojwmkdysmpy.supabase.co
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

Effort: Small (1-2 hours)

LOW F-26: No security.txt File

A05 -- Security Misconfiguration

DESCRIPTION

No `/.well-known/security.txt` file exists. Security researchers have no way to report vulnerabilities responsibly.

EVIDENCE

```
GET /.well-known/security.txt HTTP/1.1
=> HTTP 404
```

REMEDIATION

Create `/.well-known/security.txt` with contact information per RFC 9116.

Effort: Trivial (15 min)

LOW F-27: No robots.txt or sitemap.xml

A05 -- Security Misconfiguration

DESCRIPTION

Both `/robots.txt` and `/sitemap.xml` return 404. No guidance for search engine crawlers. Sensitive paths (admin, dashboard, login) are not hidden from scanners.

REMEDIATION

Create a `robots.txt` that disallows crawling of admin, dashboard, and login paths. Generate a `sitemap.xml` for public content only.

Effort: Trivial (15 min)

4.6 -- A07: Identification & Authentication Failures

HIGH **F-11: Incomplete Sign-Out -- Sensitive Data Remains (Partial Fix)** A07 -- Authentication Failures

DESCRIPTION

On sign-out, only `userEmail` and `userId` are removed from `localStorage`. The `userAvatar` key (base64 encoded profile image) is NOT removed. Other keys like `adminSettings`, `completedLessons_*`, and `moduleFeedback` also persist.

STATUS

PARTIAL FIX -- `userEmail` and `userId` are now cleared on sign-out. However, `userAvatar` is NOT removed.

EVIDENCE

```
// src/components/layout/Header.tsx (lines 280–284, 343–347)
await supabase.auth.signOut();
localStorage.removeItem('userEmail');
localStorage.removeItem('userId');
// MISSING: localStorage.removeItem('userAvatar');
// MISSING: localStorage.removeItem('completedLessons_*');
// MISSING: localStorage.removeItem('adminSettings');
```

IMPACT

On a shared computer, the next user can see the previous user's avatar, learning progress, and (if admin) all API keys.

REMEDIATION

Clear all user-specific `localStorage` keys on sign-out. Implement a utility function that enumerates and removes all application keys.

Effort: Trivial (30 min)

MEDIUM F-13: No Email Verification Enforcement After Signup

A07 -- Authentication Failures

DESCRIPTION

After signup, the user is immediately redirected to the dashboard without checking if the email has been verified. Any email address can be used to create an account.

EVIDENCE

```
// src/pages/en/login.astro (lines 260–284)
// After signup, user is redirected immediately:
window.location.href = '/en/dashboard';
// No check for: data.user?.email_confirmed_at
```

IMPACT

Users can register with any email address, enabling account impersonation and spam registrations.

REMEDIATION

Check `data.user?.email_confirmed_at` before granting access. Show a "please verify your email" page if not confirmed. Configure Supabase Auth to require email confirmation.

Effort: Small (1-2 hours)

LOW F-23: Client-Side-Only Password Validation

A07 -- Authentication Failures

DESCRIPTION

Password strength validation is performed only in JavaScript. Supabase Auth provides a server-side safety net with minimum length requirement, but no complexity enforcement.

EVIDENCE

```
// src/pages/en/login.astro (lines 218–225)
// Client-side only:
if (password.length < 6) {
  showError('Password must be at least 6 characters');
  return;
}
```

REMEDIATION

Configure Supabase Auth password policy to enforce complexity requirements server-side.

Effort: Trivial (15 min)

LOW F-24: Admin Settings Reinitialized with Hardcoded Defaults

A04 -- Insecure Design

DESCRIPTION

The `defaultSettings` object in `admin.ts` contains a hardcoded Supabase URL. When `localStorage` is empty, these defaults are returned.

EVIDENCE

```
// src/lib/admin.ts (lines 30-49)
const defaultSettings = {
  supabaseUrl: 'https://jhesstimsojwmkdysmpy.supabase.co',
  supabaseAnonKey: 'sb_publishable_...',
  revolutApiKey: '',
  // ...
};
```

REMEDIATION

Import the URL from `config.ts` instead of hardcoding it. Do not include API key placeholders in default settings.

Effort: Trivial (15 min)

LOW F-25: email_encrypted Column Stores Plaintext

A04 -- Insecure Design

DESCRIPTION

The column name `email_encrypted` is misleading -- it stores plaintext email addresses, not encrypted ones. This creates a false sense of security.

EVIDENCE

```
-- supabase/migrations/003_admin_email_access.sql
CREATE TABLE admin_user_emails (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email_encrypted TEXT NOT NULL,  -- Actually stores plaintext!
  created_at TIMESTAMPTZ DEFAULT now()
);
```

REMEDIATION

Rename the column to `email` or implement actual encryption using `pgcrypto` .

Effort: Trivial (15 min)

5. Remediation Roadmap

Issues are ordered by priority. Critical and High issues should be addressed within 1 week. Medium issues within 2-4 weeks. Low issues in the next development cycle.

Priority	Issue	Severity	Effort	Action
P1	F-01/F-02: Privilege Escalation	CRITICAL	1-2h	Restrict RLS UPDATE on profiles to prevent is_admin/plan modification
P1	F-05: Stored XSS via display_name	HIGH	1-2h	Replace innerHTML with textContent; add input validation
P1	F-06: Stored XSS via Logo URL	HIGH	30m	Use DOM APIs instead of innerHTML
P1	F-03: Admin Page No Server-Side Auth	CRITICAL	3-5h	Implement Astro middleware for admin auth checks
P1	F-04: Admin Email Exposed	CRITICAL	1-2h	Remove hardcoded email; use is_admin column
P2	F-12: Missing HTTP Headers	HIGH	1-2h	Add CSP, HSTS, X-Frame-Options, etc.
P2	F-10: Real-time IDOR	HIGH	1-2h	Derive userId from auth session, not localStorage
P2	F-11: Incomplete SignOut	HIGH	30m	Clear all user-specific localStorage keys
P2	F-07: Feature Flags Not Enforced	HIGH	2-3h	Import and check flags in all relevant pages
P2	F-09: localStorage Sensitive Data	HIGH	4-6h	Migrate to httpOnly cookies; implement CSP
P2	F-08: Base64 Avatars	HIGH	3-5h	Move to Supabase Storage
P3	F-18: Quiz Score Tampering	MEDIUM	3-5h	Use save_quiz_score RPC from migration 007
P3	F-17: Account Deletion	MEDIUM	30m	Call delete_own_account RPC from migration 007
P3	F-19: Lesson Progress Faking	MEDIUM	2-3h	Use complete_lesson RPC from migration 007
P3	F-13: No Email Verification	MEDIUM	1-2h	Check email_confirmed_at before access
P3	F-16: display_name No Validation	MEDIUM	30m	Add maxLength and DB CHECK constraint
P3	F-20: increment_points No Bounds	MEDIUM	30m	Add bounds checking in RPC
P3	F-21: Revolut Key in localStorage	MEDIUM	3-5h	Move to server-side edge function
P4	F-22: Broad delete GRANT	LOW	30m	Restrict GRANT to specific roles
P4	F-23: Client-Side Password Validation	LOW	15m	Configure Supabase Auth password policy

Priority	Issue	Severity	Effort	Action
P4	F-24: Hardcoded Defaults	LOW	15m	Import from config.ts
P4	F-25: email_encrypted Misnomer	LOW	15m	Rename column or encrypt
P4	F-26/F-27: Missing security.txt/robots.txt	LOW	15m	Create both files

6. Appendix: RLS Policy Inventory

All 5 tables in the Supabase project have RLS enabled. The following table documents every policy. Gaps are highlighted.

Table: profiles (RLS ENABLED)

Policy	Operation	Condition	Migration
Users can view own profile	SELECT	`auth.uid() = id`	001
Users can update own profile	UPDATE	`auth.uid() = id`	001
Users can insert own profile	INSERT	`auth.uid() = id`	001
Users can delete own profile	DELETE	`auth.uid() = id`	004
Admin can view all profiles	SELECT	`is_admin()`	005
Admin can update any profile	UPDATE	`is_admin()`	005
Admin can delete any profile	DELETE	`is_admin()`	005

Table: lesson_progress (RLS ENABLED)

Policy	Operation	Condition	Migration
Users can view own	SELECT	`auth.uid() = user_id`	001
Users can insert own	INSERT	`auth.uid() = user_id`	001
Users can update own	UPDATE	`auth.uid() = user_id`	001
Users can delete own	DELETE	`auth.uid() = user_id`	004
Admin can view all	SELECT	`is_admin()`	005
Admin can delete any	DELETE	`is_admin()`	005
Missing: Admin UPDATE	UPDATE	--	--

Table: quiz_scores (RLS ENABLED)

Policy	Operation	Condition	Migration
Users can view own	SELECT	`auth.uid() = user_id`	001
Users can insert own	INSERT	`auth.uid() = user_id`	001
Users can delete own	DELETE	`auth.uid() = user_id`	004
Admin can view all	SELECT	`is_admin()`	005

Policy	Operation	Condition	Migration
Admin can delete any	DELETE	`is_admin()`	005
Missing: Admin UPDATE, User UPDATE	UPDATE	--	--

Table: user_points (RLS ENABLED)

Policy	Operation	Condition	Migration
Users can view own	SELECT	`auth.uid() = user_id`	001
Users can update own	UPDATE	`auth.uid() = user_id`	001
Users can insert own	INSERT	`auth.uid() = user_id`	001
Users can delete own	DELETE	`auth.uid() = user_id`	004
Admin can view all	SELECT	`is_admin()`	005
Admin can delete any	DELETE	`is_admin()`	005
Missing: Admin UPDATE	UPDATE	--	--

Table: admin_user_emails (RLS ENABLED)

Policy	Operation	Condition	Migration
Admin only access	ALL	`is_admin()`	005

Tables WITHOUT RLS: None. All 5 tables have RLS enabled.

Policies with `true` condition: None found.

RLS Gaps Summary

Table	Missing Policies	Impact
lesson_progress	Admin UPDATE	Admin cannot correct user progress
quiz_scores	Admin UPDATE, User UPDATE	No score correction or retake mechanism
user_points	Admin UPDATE	Admin cannot adjust user points
profiles	Column-level restriction on is_admin, plan	Users can self-promote (F-01, F-02)